

PROMPT KIMI K3 — WALLET APP v3.1

7 funcionalidades faltantes para ser #1 no mercado

ECONOMIA DE TOKENS: referenciar arquivos existentes, nao colar codigo

=====

CONTEXTO DO PROJETO (ja existe, NAO recriar)

Repo: heitorfragavd-prog/wallet-App | Branch: develop Stack: React 18 + TS + Vite + Supabase + shadcn/ui + Tailwind + Recharts

Arquitetura por dominios: src/domains/{auth,finance,eyemobile,vehicles,market}/ Hooks: usam @tanstack/react-query (padrao: useQuery + useMutation + invalidate) RLS: todas as tabelas tem RLS por user_id Workspace: PF/PJ via WorkspaceContext.tsx

ARQUIVOS EXISTENTES (copiar padrao deles):

- Hook modelo: src/domains/finance/hooks/useDespesas.ts
- Hook modelo: src/domains/finance/hooks/useFoodCost.ts
- Pagina modelo: src/pages/Cardapio.tsx
- Pagina modelo: src/pages/Validades.tsx
- Edge Function modelo: supabase/functions/eyemobile-sync/index.ts
- Edge Function modelo: supabase/functions/categorizar-ia/index.ts
- Tipagem modelo: src/domains/finance/types/foodCost.ts
- App.tsx: ja tem lazyWithRetry e ProtectedRoute
- Menu: DashboardLayout.tsx ou Sidebar (adicionar itens no array)

REGRAS:

1. SEMPRE filtrar por workspace_id
2. SEMPRE usar React Query (useQuery/useMutation)
3. SEMPRE usar toast para feedback
4. SEMPRE usar shadcn/ui components
5. SEMPRE adicionar RLS em novas tabelas
6. NUNCA sobrescrever hooks/paginas existentes que funcionam
7. Preservar todos os testes *.test.ts

TAREFA: IMPLEMENTAR 7 FUNCIONALIDADES

1. SUBCATEGORIAS

SQL (migration unica no final)

```
CREATE TABLE subcategorias (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id uuid REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,  
  workspace_id uuid REFERENCES workspaces(id) ON DELETE CASCADE,  
  categoria_id uuid REFERENCES categorias(id) ON DELETE CASCADE,  
  nome text NOT NULL, cor text DEFAULT '#3B82F6', ativo boolean DEFAULT true,  
  created_at timestamptz DEFAULT now()  
);  
  
ALTER TABLE subcategorias ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "Users can manage their own subcategorias" ON subcategorias FOR ALL USING (auth.uid() =  
  ⇨ user_id);  
CREATE INDEX idx_subcategorias_workspace ON subcategorias(workspace_id);  
CREATE INDEX idx_subcategorias_categoria ON subcategorias(categoria_id);  
  
ALTER TABLE despesas ADD COLUMN IF NOT EXISTS subcategoria_id uuid REFERENCES subcategorias(id);  
ALTER TABLE receitas ADD COLUMN IF NOT EXISTS subcategoria_id uuid REFERENCES subcategorias(id);
```

Arquivos a criar

- src/domains/finance/hooks/useSubcategorias.ts (copiar padrao de useDespesas.ts)
- src/pages/Subcategorias.tsx (copiar padrao de Cardapio.tsx — lista CRUD)

Modificar

- Despesas.tsx: no formulario, adicionar Select de subcategoria (filtra por categoria selecionada)
- Receitas.tsx: idem

2. TRANSFERENCIAS ENTRE CONTAS

SQL

```
CREATE TABLE transferencias (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id uuid REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,  
  workspace_id uuid REFERENCES workspaces(id) ON DELETE CASCADE,  
  conta_origem_id uuid REFERENCES contas_usuario(id) ON DELETE CASCADE NOT NULL,  
  conta_destino_id uuid REFERENCES contas_usuario(id) ON DELETE CASCADE NOT NULL,  
  valor numeric(10,2) NOT NULL, data date NOT NULL,  
  descricao text, observacoes text, created_at timestamptz DEFAULT now()  
);  
  
ALTER TABLE transferencias ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "Users can manage their own transferencias" ON transferencias FOR ALL USING (auth.uid() =  
  ⇨ user_id);
```

Arquivos a criar

- src/domains/finance/hooks/useTransferencias.ts (copiar padrao useDespesas.ts)
 - useQuery busca transferencias do workspace
 - useMutation create: insere na tabela + atualiza saldo conta origem (diminui) + destino (aumenta)
- src/pages/Transferencias.tsx (copiar padrao Cardapio.tsx)
 - Lista de transferencias
 - Formulário: Select conta origem, Select conta destino, Input valor, Input data, Input descricao
 - NAO permitir conta_origem === conta_destino

Modificar

- App.tsx: adicionar rota /transferencias
- Menu: adicionar item Transferencias (icone ArrowRightLeft)

3. RECIBOS

Edge Function

- supabase/functions/gerar-recibo/index.ts (copiar padrao de categorizar-ia)
 - Recebe: valor, pagador, recebedor, descricao, data, cidade
 - Retorna HTML formatado de recibo com assinatura
 - Usar template simples com CSS inline

Pagina

- src/pages/Recibos.tsx
 - Formulário: valor, pagador, recebedor, descricao, data, cidade
 - Botao “Gerar Recibo” → chama Edge Function → mostra preview HTML
 - Botao “Baixar PDF” → usa window.print() ou jsPDF (se ja instalado)

Modificar

- App.tsx: rota /recibos
- Menu: item Recibos (icone Receipt)

4. CONCILIAÇÃO BANCARIA

Hook

- src/domains/finance/hooks/useConciliacao.ts (copiar padrao useDespesas.ts)
 - Busca receitas + despesas do mes (por workspace)
 - Busca transacoes do Open Finance/Pluggy do mes (se disponivel)
 - Retorna: pendentes (lancados no app mas nao no banco) + conciliados
 - Por enquanto, mockar dados do banco ou usar transacoes da tabela transacoes

Pagina

- src/pages/Conciliacao.tsx
 - Tabela lado a lado: Lancamentos do App vs Extrato Bancario
 - Checkbox para marcar como conciliado

- Badge: Pendente / Conciliado
- Botao “Conciliar automaticamente” (match por valor + data aproximada)

Modificar

- App.tsx: rota /conciliacao
- Menu: item Conciliacao (icone CheckCircle)

5. CENTRO DE CUSTO

SQL

```
CREATE TABLE centros_custo (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  workspace_id uuid REFERENCES workspaces(id) ON DELETE CASCADE,
  nome text NOT NULL, descricao text, responsavel text,
  orcamento_mensal numeric(10,2), ativo boolean DEFAULT true,
  created_at timestamptz DEFAULT now()
);
ALTER TABLE centros_custo ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage their own centros_custo" ON centros_custo FOR ALL USING (auth.uid() =
↪ user_id);

ALTER TABLE despesas ADD COLUMN IF NOT EXISTS centro_custo_id uuid REFERENCES centros_custo(id);
ALTER TABLE receitas ADD COLUMN IF NOT EXISTS centro_custo_id uuid REFERENCES centros_custo(id);
```

Arquivos a criar

- src/domains/finance/hooks/useCentrosCusto.ts (padrao useDespesas.ts)
- src/pages/CentrosCusto.tsx (padrao Cardapio.tsx — lista CRUD)

Modificar

- Despesas.tsx: Select de centro de custo no formulario
- Receitas.tsx: idem
- App.tsx: rota /centros-custo
- Menu: item Centros de Custo (icone Building)

6. FORNECEDORES / CLIENTES (CRM BASICO)

SQL

```
CREATE TABLE contatos (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  workspace_id uuid REFERENCES workspaces(id) ON DELETE CASCADE,
  tipo text NOT NULL CHECK (tipo IN ('fornecedor', 'cliente')),
  nome text NOT NULL, cnpj_cpf text, telefone text, email text,
  endereco text, contato_nome text, prazo_pagamento_dias integer DEFAULT 30,
  observacoes text, created_at timestamptz DEFAULT now()
);
```

```

ALTER TABLE contatos ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage their own contatos" ON contatos FOR ALL USING (auth.uid() = user_id);
CREATE INDEX idx_contatos_tipo ON contatos(tipo);
CREATE INDEX idx_contatos_workspace ON contatos(workspace_id);

ALTER TABLE despesas ADD COLUMN IF NOT EXISTS contato_id uuid REFERENCES contatos(id);
ALTER TABLE receitas ADD COLUMN IF NOT EXISTS contato_id uuid REFERENCES contatos(id);

```

Arquivos a criar

- src/domains/finance/hooks/useContatos.ts (padrao useDespesas.ts)
- src/pages/Fornecedores.tsx (padrao Cardapio.tsx)
 - Tabs: Fornecedores / Clientes
 - Cards com: nome, tipo, telefone, email, prazo pagamento
 - Botao Novo Contato

Modificar

- Despesas.tsx: Select de fornecedor no formulario
- Receitas.tsx: Select de cliente no formulario
- App.tsx: rota /fornecedores
- Menu: item Fornecedores (icone Users)

7. AGENDA FINANCEIRA

Pagina

- src/pages/Agenda.tsx (copiar padrao de Validades.tsx)
 - Usar componente Calendar do shadcn/ui
 - Selecionar dia → mostra compromissos daquele dia
 - Compromissos: despesas (data) + dividas (data_vencimento) + receitas (data)
 - Cores: verde = receita, vermelho = despesa, amarelo = divida
 - Badge com contador de compromissos no dia

Modificar

- App.tsx: rota /agenda
- Menu: item Agenda (icone Calendar)

MIGRACAO SQL UNICA

Criar arquivo: supabase/migrations/20250802_features_v31.sql

Juntar TODOS os CREATE TABLE e ALTER TABLE acima em um unico arquivo.

MODIFICACOES EM ARQUIVOS EXISTENTES

App.tsx

Adicionar imports lazy:

```
const Transferencias = lazyWithRetry(() => import("./pages/Transferencias"));
const Agenda = lazyWithRetry(() => import("./pages/Agenda"));
const Subcategorias = lazyWithRetry(() => import("./pages/Subcategorias"));
const CentrosCusto = lazyWithRetry(() => import("./pages/CentrosCusto"));
const Fornecedores = lazyWithRetry(() => import("./pages/Fornecedores"));
const Conciliacao = lazyWithRetry(() => import("./pages/Conciliacao"));
const Recibos = lazyWithRetry(() => import("./pages/Recibos"));
```

Adicionar rotas protegidas:

```
<Route path="/transferencias" element={<ProtectedRoute><Transferencias /></ProtectedRoute>} />
<Route path="/agenda" element={<ProtectedRoute><Agenda /></ProtectedRoute>} />
<Route path="/subcategorias" element={<ProtectedRoute><Subcategorias /></ProtectedRoute>} />
<Route path="/centros-custo" element={<ProtectedRoute><CentrosCusto /></ProtectedRoute>} />
<Route path="/fornecedores" element={<ProtectedRoute><Fornecedores /></ProtectedRoute>} />
<Route path="/conciliacao" element={<ProtectedRoute><Conciliacao /></ProtectedRoute>} />
<Route path="/recibos" element={<ProtectedRoute><Recibos /></ProtectedRoute>} />
```

Menu lateral (DashboardLayout.tsx ou Sidebar)

Adicionar itens:

```
{ title: "Transferencias", href: "/transferencias", icon: ArrowRightLeft },
{ title: "Agenda", href: "/agenda", icon: Calendar },
{ title: "Subcategorias", href: "/subcategorias", icon: ListTree },
{ title: "Centros de Custo", href: "/centros-custo", icon: Building },
{ title: "Fornecedores", href: "/fornecedores", icon: Users },
{ title: "Conciliacao", href: "/conciliacao", icon: CheckCircle },
{ title: "Recibos", href: "/recibos", icon: Receipt },
```

Importar icones do lucide-react: ArrowRightLeft, Calendar, ListTree, Building, Users, CheckCircle, Receipt

Despesas.tsx e Receitas.tsx

Adicionar nos formulários:

- Select subcategoria (filtrar por categoria selecionada)
- Select centro_custo
- Select fornecedor/cliente (contato)

CHECKLIST DE ENTREGA

- 7 hooks criados em `src/domains/finance/hooks/`
- 7 paginas criadas em `src/pages/`
- 1 Edge Function criada em `supabase/functions/gerar-recibo/`
- 1 migration SQL em `supabase/migrations/20250802__features__v31.sql`
- `App.tsx` com 7 novas rotas
- Menu lateral com 7 novos itens
- `Despesas.tsx` e `Receitas.tsx` com novos Selects
- TypeScript: 0 erros
- Build: sucesso
- Nenhum arquivo existente quebrado